

Transformer 学习资料（教材式版本）

目标读者：机器学习初学者、自动驾驶端到端入门者

阅读目标：读完后能独立读懂 Transformer / Attention 基本原理，并能看懂自动驾驶论文里的核心模块

目录

文档一：Transformer 与 Attention 从零到一

1. 为什么需要 Attention
2. Attention 的数学定义与直觉
3. Q / K / V 的作用
4. Self-Attention
5. Multi-Head Attention
6. FFN（前馈网络）
7. Transformer Encoder 的完整结构
8. 一个完整自动驾驶例子

文档二：官方论文《Attention Is All You Need》系统解读

1. 论文背景
 2. 核心思想
 3. Encoder-Decoder 架构
 4. Positional Encoding
 5. Scaled Dot-Product Attention
 6. 论文的历史意义
-

文档一：Transformer 与 Attention 从零到一

第 1 章 为什么需要 Attention

官方解释

在序列建模中，输入通常由多个元素组成：

$$X = [x_1, x_2, \dots, x_n]$$

传统神经网络很难动态判断：

当前任务最重要的信息来自哪里。

Attention 被提出，用来显式建模输入元素之间的依赖关系。

小白解释

自动驾驶场景里：

你前方可能有：

- 前车
- 左侧车
- 红灯
- 行人

真正关键的问题不是“看见多少东西”。

而是：

当前哪一个对象最影响你的决策？

Attention 就是干这个的。

第 2 章 Attention 的数学定义

官方解释

给定输入矩阵：

$$X \in \mathbb{R}^{n \times d}$$

经过三组线性映射：

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

Attention 定义为：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

其中：

第一步

$$QK^T$$

计算 token 之间相关性。

第二步

softmax 将相关性转成概率权重。

第三步

用权重加权融合 Value。

小白解释

Attention 本质只做两件事：

第一步

决定应该重点关注谁。

第二步

把关注对象的信息拿回来。

一句话：

Attention = 决定看谁 + 拿回信息

第 3 章 Q / K / V 到底是什么

官方解释

Transformer 不直接使用原始输入。

而是学习三种不同表示：

- Query
- Key
- Value

作用如下：

Query

表示当前 token 想获取的信息类型。

Key

表示当前 token 可以被匹配的属性。

Value

表示当前 token 被关注后真正输出的内容。

小白解释

你可以把它想成你在开车。

Query

你在想：

我现在最担心什么？

比如：前方风险。

Key

周围车辆在“展示自己”：

我是不是你当前应该关注的对象？

Value

如果你决定关注它，

你真正拿回来的信息是什么？

比如：

- 它在减速
 - 它在靠近
-

第 4 章 Self-Attention

官方解释

Self-Attention 指：

输入序列中每个 token 都可以访问整个序列。

因此：

每个 token 在更新时都可以结合其它 token 的信息。

小白解释

更新 ego 时，ego 可以同时看：

- 前车
- 左车
- 红灯
- 车道线

所以 Self-Attention 的核心价值不是识别对象。

而是：

理解对象之间的关系。

第 5 章 Multi-Head Attention

官方解释

单个 Attention 往往只能学习一种关系。

因此引入多个独立 head：

$$head_i = Attention(Q_i, K_i, V_i)$$

最后拼接：

$$MultiHead = Concat(head_1, ..., head_h)W_O$$

作用：

在不同子空间学习不同关系。

小白解释

自动驾驶里可能同时存在：

- 前车减速
- 左车 cut-in
- 红灯限制

多个 head 可以理解成多个观察员：

head 1

盯前车

head 2

盯左车

head 3

盯红灯

最后综合所有观察结果。

第 6 章 FFN（前馈网络）

官方解释

每个 Transformer block 中包含：

$$FFN(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

其作用是：

对每个 token 独立进行非线性特征变换。

小白解释

Attention 是：

从别人那里收集信息。

FFN 是：

自己消化这些信息。

比如：

Attention 后 ego 知道：

- 前车减速
- 左车靠近

FFN 后 ego 可能进一步形成：

当前风险变大。

第 7 章 Transformer Encoder 的完整结构

官方解释

标准 Encoder Block:

1. Multi-Head Self-Attention
2. Residual + LayerNorm
3. Feed Forward Network
4. Residual + LayerNorm

多个 block 堆叠后形成深层表示。

小白解释

每一层都在做两件事:

第一步

看别人, 收集信息。

第二步

自己整理这些信息。

层数越深, 语义越高。

例如:

第一层

知道前车近。

第二层

知道左车可能 cut-in。

第三层

知道应该减速。

第 8 章 自动驾驶完整例子 (最重要)

我们用一个完整例子把前面所有内容串起来。

场景

你 (ego) 正在直行。

周围有：

- 前车减速
- 左车靠近

设三个 token：

token	特征
ego	[1, 0]
front	[3, 0]
left	[0, 2]

含义：

- 第 1 维：纵向风险
- 第 2 维：横向风险

第一步：计算相关性

设 ego 当前 query：

$$q = [1, 0]$$

计算：

$$q \cdot ego = 1$$

$$q \cdot front = 3$$

$$q \cdot left = 0$$

得到：

$$score = [1, 3, 0]$$

说明：

ego 当前最关注前车。

第二步：softmax

近似得到：

对象	权重
ego	0.11
front	0.84
left	0.05

第三步：融合信息

$$z_{ego} = 0.11 \cdot ego + 0.84 \cdot front + 0.05 \cdot left$$

结果：

$$z_{ego} \approx [2.63, 0.10]$$

小白解释

更新后的 ego 已经不再只是 ego 自己。

它已经带上了：

- 强烈的前车减速信息
- 少量左车风险信息

这就是 Attention 真正在做的事情。

文档二：官方论文《Attention Is All You Need》系统解读

第 1 章 论文背景

官方解释

2017 年，Vaswani 等人在论文《Attention Is All You Need》中提出 Transformer。

目标是解决：

1. RNN 无法高效并行计算
2. 长距离依赖难以建模

小白解释

以前模型像读书：

必须一个字一个字往后看。

Transformer 像：

一眼看全句。

第 2 章 核心思想

官方解释

论文提出：

完全基于 Attention 的序列建模框架。

不再依赖 RNN / LSTM。

小白解释

以前靠“记忆”。

Transformer 靠：

直接建立全局关系。

第 3 章 Encoder-Decoder 架构

官方解释

Encoder

每层包括：

- Multi-Head Self-Attention
- Feed Forward Network

Decoder

比 Encoder 多一个：

- Encoder-Decoder Attention

小白解释

Encoder:

理解输入。

Decoder:

根据理解生成输出。

第 4 章 Positional Encoding

官方解释

由于 Attention 本身不含顺序信息，论文加入位置编码：

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

小白解释

Transformer 自己不知道：

- 谁在前
- 谁在后

所以必须额外告诉它位置。

第 5 章 为什么除以 $\sqrt{d_k}$

官方解释

点积维度越高，数值越大。

为了稳定 softmax：

$$\frac{QK^T}{\sqrt{d_k}}$$

小白解释

就是为了防止数值太大，把 softmax 弄得过于极端。

第 6 章 论文真正改变了什么

官方解释

Transformer 将序列建模从：

递归状态传播

转变为：

全局依赖关系建模

小白解释

以前模型：

一步一步记住过去。

Transformer：

直接建立全局关系。